Symbiosis Institute of Technology, Nagpur

BIG DATA ANALYTICS MINI PROJECT REPORT

for

Bachelor of Technology
in
Computer Science and Engineering
Batch 2023-27
on

Fake News Detection using Big Data Analysis

Submitted by

Adhikar Shende – 23070521236
Kshiteej Waghdeve – 23070521172

Semester: VII
Course: Big Data Analytics
Course Code: 0705210731

Industry Guide
Dr. Dharmaveer Chaudhary
Institute Guide
Dr. Pratik Agrawal

Academic Year: 2026-27
Date: 17th October, 2026

॥वसुधैव कुटुम्बकम्॥

Symbiosis International (Deemed University), Pune

ABSTRACT

The rapid spread of fabricated news on digital platforms misleads readers, distorts public debate and erodes trust in genuine journalism, yet the volume of content published every day makes manual fact-checking impossible. This project builds a scalable text-classification pipeline that separates fake news articles from real ones using big data analysis techniques. The ISOT fake and real news dataset of 44,898 articles (23,481 fake and 21,417 real) was merged, labelled and shuffled, and the title and body of each article were combined into a single text field. The data was split 80/20 into 35,918 training and 8,980 test articles. Text was converted into a sparse numerical representation using TF-IDF vectorisation with English stop-word removal, and three classifiers suited to high-dimensional sparse text were compared: a Passive-Aggressive Classifier, Logistic Regression and Multinomial Naive Bayes. Because the two classes are nearly balanced, accuracy is a meaningful metric and is reported together with precision, recall, F1-score and the confusion matrix. On the held-out test set the Passive-Aggressive Classifier achieved the best accuracy of 99.44 percent with only 50 misclassified articles, followed by Logistic Regression at 98.45 percent and Naive Bayes at 93.37 percent. The work demonstrates that an online-learning linear model on TF-IDF features is fast, memory-efficient and highly accurate for fake news detection, with the caveat that scores on a single-domain benchmark dataset are optimistic.

Keywords: *Fake News Detection, Big Data, Text Mining, TF-IDF, Passive-Aggressive Classifier, Logistic Regression, Naive Bayes*

1. INTRODUCTION

1.1 Background

Social media, messaging applications and online news portals now deliver information to billions of people in real time. The same channels that make information accessible also allow false stories to be created cheaply and to spread faster and further than accurate ones [1], [2]. Fake news has been linked to political manipulation, public-health misinformation, financial scams and social unrest, and every false story that is believed damages trust in credible media organisations.

Fake news detection is a natural big data problem. Millions of articles, posts and headlines are produced every day, which rules out manual verification and demands automated methods that can process large volumes of unstructured text. Text must be converted into numerical features, usually producing very high-dimensional but sparse matrices, so memory-efficient representations and scalable learning algorithms are central engineering concerns [3], [5].

1.2 Motivation

Fact-checking organisations can verify only a tiny fraction of the claims that circulate online, and verification usually happens after a story has already gone viral. Automated classifiers can screen content at scale and prioritise suspicious items for human reviewers. This project is motivated by the need to understand how well classical big data text analytics, which are cheap to train and easy to deploy, can separate fake from real news, and how the choice of learning algorithm affects the result.

1.3 Problem Statement

Given a large collection of news articles labelled as fake or real, build and evaluate a machine learning pipeline that converts raw article text into numerical features using scalable text-mining techniques and classifies each article as FAKE or REAL with high accuracy, using evaluation measures that expose both kinds of error: real news wrongly flagged as fake and fake news wrongly accepted as real.

1.4 Objectives

1. Load, merge, label and profile a large news dataset of 44,898 articles.
2. Visualise the class balance and the distribution of articles across news subjects.
3. Prepare the text and split it into training and testing sets.
4. Convert text into sparse numerical features using TF-IDF vectorisation.
5. Train and compare a Passive-Aggressive Classifier, Logistic Regression and Multinomial Naive Bayes.

6. Evaluate the models using accuracy, precision, recall, F1-score and the confusion matrix, and identify the best model.

1.5 Novelty of the Work

- Three different learning paradigms (online margin-based learning, probabilistic linear classification and generative probabilistic classification) are compared on identical TF-IDF features and an identical split.
- The Passive-Aggressive Classifier, an online learning algorithm designed for streaming and large-scale text, is used as the primary model, which suits the big data setting where articles arrive continuously.
- The article title is combined with the body text so that headline cues, which are known to differ between fake and real news [9], are available to the model.
- The pipeline relies on sparse matrices and linear models, so the same design can be moved to Spark MLlib or an incremental learning set-up when data grows beyond a single machine.

2. LITERATURE REVIEW / RELATED WORK

2.1 Overview of Existing Methods

Early work on deception and misinformation relied on hand-crafted linguistic cues and manual fact-checking. Surveys of the field describe fake news detection as a data mining problem that can use news content, the social context in which a story spreads, or both [1]. Content-based approaches extract lexical, syntactic and stylistic features from the text, and several studies show that fake and real articles differ in word choice, readability, punctuation and headline style [8], [9].

Supervised machine learning is the dominant approach. Word n-gram and TF-IDF representations [5] combined with linear classifiers such as Support Vector Machines, Logistic Regression and Naive Bayes [3], [12] remain strong baselines, because text classification problems are high-dimensional and often close to linearly separable. Online learners such as the Passive-Aggressive algorithm [4] update the model one example at a time and are well suited to large and streaming text collections. More recently, deep language models such as BERT [11] have been applied to fake news detection, at a much higher computational cost. Benchmark datasets such as LIAR [10] and the ISOT dataset [3] have made comparisons reproducible, and distributed frameworks such as Apache Spark [13] allow the same pipelines to be scaled out across clusters.

2.2 Comparison and Limitations

Simple linear models are fast, transparent and need little memory, but they treat words independently and ignore word order and context. Deep models capture context but need large labelled corpora and substantial hardware. Many reported results on single-source benchmarks are very high because fake and real articles in the same dataset often differ in publisher-specific style, so the scores may not carry over to articles from other sources or time periods. Content-only methods also cannot judge the factual truth of a claim, only whether the writing resembles the examples seen in training.

Table 1. Empirical review of existing methods

Reference	Method Used	Findings	Results	Limitations
Shu et al. [1] (2017)	Survey of fake news detection as a data mining problem	Organises detection methods by news content and social context and lists open challenges	Taxonomy of features, models and datasets	Survey only; no new experimental results
Vosoughi et al. [2] (2018)	Large-scale analysis of rumour cascades on Twitter	False news diffuses farther, faster and more broadly than true news	Strong evidence that false stories spread faster	Studies spread, not automatic classification
Ahmed et al. [3] (2017)	TF-IDF n-grams with linear SVM and other classifiers	Introduced the ISOT dataset; linear classifiers work well on n-gram features	Reported accuracy of up to about 92 percent	Single-source data; content features only
Crammer et al. [4] (2006)	Online Passive-Aggressive algorithms	Margin-based updates that change the model only when an example is misclassified	Competitive accuracy with very fast training	Sensitive to noisy labels and learning-rate settings
Perez-Rosas et al. [8] (2018)	Linguistic feature analysis with linear SVM	Fake and real news differ in lexical and syntactic features	Good accuracy within the same domain	Performance falls across domains
Horne and Adali [9] (2017)	Stylistic and complexity features	Fake headlines are longer and fake body text is simpler and more repetitive	Features separate fake from real news well	Small datasets; style may reflect the source
Wang [10] (2017)	LIAR benchmark with hybrid neural models	Provides about 12,800 fact-checked short statements with six-way labels	Metadata combined with text improves results	Short statements; accuracy remains modest
Devlin et al. [11] (2019)	BERT pre-trained language model	Contextual embeddings improve many text classification tasks	State-of-the-art on many language benchmarks	High computational cost; harder to scale and explain

2.3 Research Gap

The reviewed work leaves three gaps that this project addresses. First, many studies report a single model, so there is little side-by-side evidence of how online, discriminative and generative linear models behave on the same features and split. Second, fake news detection is rarely presented as a big data pipeline with attention to sparse representations and scalability. Third, results are often reported as accuracy alone without the confusion matrix and class-wise precision and recall that show which kind of error a model makes. This project compares three algorithms under identical conditions and reports class-wise measures.

3. METHODOLOGY / PROPOSED SYSTEM

3.1 Approach and Workflow

The proposed system is a batch pipeline from raw article files to evaluated classifiers. Fig. 1 summarises the workflow. The key design rules are that the TF-IDF vocabulary and weights are learned from the training split only and then applied unchanged to the test split, that all three models use the same features and the same split, and that performance is reported on articles the models never saw during training.

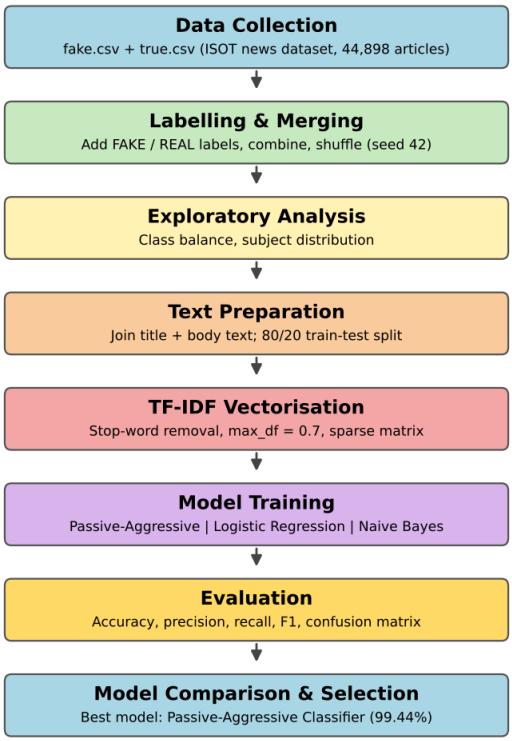


Fig. 1. System architecture and workflow

3.2 Data Collection and Preprocessing

The ISOT Fake and Real News dataset, distributed on Kaggle as two files, fake.csv and true.csv, was used [3], [7]. The real articles were collected from Reuters.com and the fake articles from sites flagged as unreliable by fact-checking organisations. The dataset was chosen because of its size (44,898 articles), its nearly balanced classes, the availability of both a title and a full text for each article, and because it is public and reproducible.

Table 2. Dataset overview

Property	Value
Files	fake.csv and true.csv, merged into one data frame
Total articles / Columns	44,898 / 5 (title, text, subject, date, label)
Fake articles	23,481 (52.3 percent)
Real articles	21,417 (47.7 percent)
Subjects	politicsNews, worldnews (real); News, politics, left-news, Government News, US_News, Middle-east (fake)
Label encoding	FAKE and REAL added as a new label column

After loading, the two files were given the labels FAKE and REAL, concatenated, and shuffled with a fixed random state of 42 so that the order of rows carries no information. The title and the text of each article were joined into one field, total_text, which serves as the model input. The subject and date columns were used only for exploratory analysis and were not given to the models, because the subject names are assigned by source and would let a classifier identify the class without reading the article. No other cleaning was applied; stop-word removal and weighting are handled inside the vectoriser.

3.3 Feature Engineering

Text was converted to numbers with Term Frequency - Inverse Document Frequency (TF-IDF) [5]. Each article becomes a sparse vector in which every dimension is a vocabulary word, weighted by how often it appears in the article and how rare it is across all articles:

tf-idf(t, d) = tf(t, d) x idf(t)
idf(t) = log((1 + N) / (1 + df(t))) + 1
N = number of training articles, df(t) = articles containing term t

Common English stop words such as "the" and "and" were removed, and terms appearing in more than 70 percent of the documents (max_df = 0.7) were ignored because they do not help separate articles. The vectoriser was fitted on the training articles only. The resulting matrix is stored in sparse format, so only non-zero entries use memory, which is what makes the approach practical for large text collections.

3.4 Model Design

Three classifiers were chosen to represent different learning paradigms, all of which work directly on sparse TF-IDF vectors.

- **Passive-Aggressive Classifier (PAC).** An online margin-based linear classifier [4]. It stays passive when an example is classified correctly with sufficient margin and becomes aggressive, updating the weights just enough to correct the mistake, when it

is not. It trains quickly and needs little memory, which suits large text streams. Settings: max_iter = 50, random_state = 42.

- **Logistic Regression (LR).** A discriminative linear model that estimates the probability that an article is real or fake. Settings: max_iter = 1,000, random_state = 42.
- **Multinomial Naive Bayes (NB).** A generative probabilistic model that assumes words occur independently given the class [12]. It is extremely fast and serves as a lightweight baseline.

3.5 Training and Evaluation Setup

The merged data was split 80/20 into a training set and a test set with a fixed random state of 42. All feature extraction was fitted on the training set, and all reported results come from the untouched test set.

Table 3. Data splits

Split	Articles	Share of data	Purpose
Train	35,918	80%	Fit TF-IDF and the classifiers
Test	8,980	20%	Final evaluation (4,710 fake, 4,270 real)
Total	44,898	100%	

Because the two classes are nearly balanced (52.3 percent against 47.7 percent), accuracy is a meaningful headline metric here, unlike in rare-event problems. It is still reported together with measures that show the type of error:

- **Accuracy** = (TP + TN) / total: the share of all articles classified correctly.
- **Precision** = TP / (TP + FP): of the articles predicted as a class, the share that truly belong to it.
- **Recall** = TP / (TP + FN): of the articles that truly belong to a class, the share that was found.
- **F1-score** = 2PR / (P + R): the harmonic mean of precision and recall.
- **Confusion matrix**: counts of correct and incorrect predictions for each class, showing real news flagged as fake and fake news accepted as real.

3.6 Pseudocode

```
1. fake <- read fake.csv;  real <- read true.csv
2. fake.label <- "FAKE";  real.label <- "REAL"
3. df <- concatenate(fake, real);  shuffle(df, seed = 42)
4. df.total_text <- df.title + " " + df.text
```

5. split (total_text, label) 80/20 -> train, test
6. tfidf <- TfidfVectorizer(stop_words = english, max_df = 0.7)
7. Xtrain <- tfidf.fit_transform(train); Xtest <- tfidf.transform(test)
8. for each model m in {PassiveAggressive, LogisticRegression, NaiveBayes}:
   fit m on (Xtrain, ytrain)
   predict ytest_hat <- m(Xtest)
   compute accuracy, precision, recall, F1, confusion matrix
9. select the model with the highest test accuracy

3.7 Tools and Libraries

Python 3 was used with pandas and NumPy for data handling, scikit-learn [6] for TF-IDF vectorisation, the three classifiers, the train-test split and the evaluation metrics, and matplotlib and seaborn for visualisation. The work was carried out in a Google Colab notebook.

4. IMPLEMENTATION

4.1 Implementation Steps

1. Environment set-up: pandas, numpy, matplotlib, seaborn and scikit-learn are imported, and a fixed random state of 42 is used for reproducibility.
2. Data loading and labelling: fake.csv and true.csv are read into data frames, labelled FAKE and REAL, concatenated into 44,898 rows and shuffled.
3. Exploratory analysis: the class distribution and the number of articles per subject are plotted (Section 5.2).
4. Preprocessing and splitting: the title and text are joined into total_text and the data is split 80/20 into 35,918 training and 8,980 test articles.
5. Vectorisation: a TfidfVectorizer with English stop-word removal and max_df = 0.7 is fitted on the training text and applied to the test text.
6. Model training: the Passive-Aggressive Classifier (max_iter = 50) is trained first, followed by Logistic Regression (max_iter = 1,000) and Multinomial Naive Bayes on the same vectors.
7. Evaluation: accuracy, the classification report and the confusion matrix are computed on the test set, and the accuracies of the three models are compared in a bar chart.

4.2 Technologies and Platforms

The implementation uses Python in a Google Colab notebook. Libraries are listed in Section 3.7. TF-IDF features are held as a scipy sparse matrix so that the memory used depends on the number of non-zero entries rather than on the full articles-by-vocabulary size.

4.3 Challenges Faced and How They Were Handled

Table 4. Challenges and solutions

Challenge	How it was handled
High-dimensional, unstructured text	TF-IDF vectorisation with stop-word removal and max_df filtering, stored as a sparse matrix
Memory and training time on a large corpus	Linear models that train directly on sparse data; online Passive-Aggressive learning
Information leakage between train and test data	Split performed before vectorisation; vocabulary fitted on the training split only
Source-specific cues (for example subject names)	Subject and date used only for analysis; models receive only the title and text
Fair comparison of algorithms	Same features, same split and same random state for all three models
Scaling beyond one machine	The same pipeline can be moved to Spark MLlib or incremental learning (future work)

4.4 Sample Outputs

Selected console output from the notebook run is reproduced below.

```
Total dataset shape: (44898, 5)
Train set size: 35918, Test set size: 8980
Accuracy: 99.44%
Logistic Regression Accuracy: 98.45%
Multinomial Naive Bayes Accuracy: 93.37%
```

5. RESULTS AND DISCUSSION

5.1 Experimental Setup

All experiments were run in a Python notebook on Google Colab using the libraries in Section 3.7, with random state 42. Hardware specification: [to be filled in by the author, for example the Colab runtime type, CPU and RAM]. Final metrics are computed once on the test set of 8,980 articles containing 4,710 fake and 4,270 real articles.

5.2 Exploratory Data Analysis

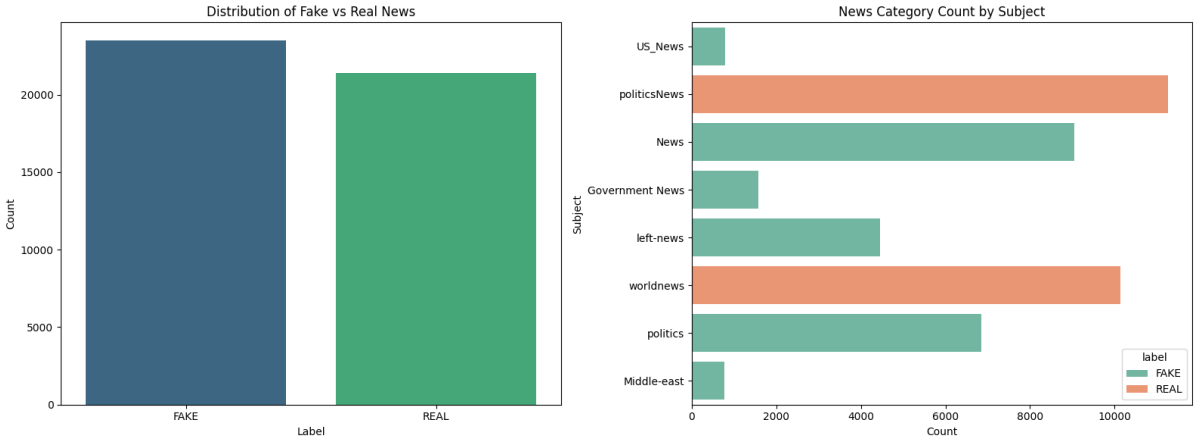


Fig. 2. Distribution of fake vs real news and article count by subject

Fig. 2 (left) shows that the dataset is nearly balanced: 23,481 fake and 21,417 real articles, so no resampling was required and accuracy is an appropriate metric. Fig. 2 (right) shows the number of articles per subject. Real articles appear only under politicsNews and worldnews, while fake articles are spread across News, politics, left-news, Government News, US_News and Middle-east, so the subject labels perfectly separate the classes. For this reason the subject column was excluded from the model input.

Table 5. Articles per subject and class

Subject	Class	Articles
politicsNews	REAL	11,272
worldnews	REAL	10,145
News	FAKE	9,050
politics	FAKE	6,841
left-news	FAKE	4,459
Government News	FAKE	1,570
US_News	FAKE	783
Middle-east	FAKE	778

5.3 Train-Test Outcome

Table 6. Class counts in the evaluation data

Stage	Fake	Real	Total
Full dataset	23,481	21,417	44,898
Training split (80%)	not printed	not printed	35,918
Test split (20%)	4,710	4,270	8,980

5.4 Model Performance

Table 7. Test-set accuracy of the three models

Model	Learning type	Test accuracy	Rank
Passive-Aggressive Classifier	Online, margin-based	99.44%	1
Logistic Regression	Discriminative, probabilistic	98.45%	2
Multinomial Naive Bayes	Generative, probabilistic	93.37%	3

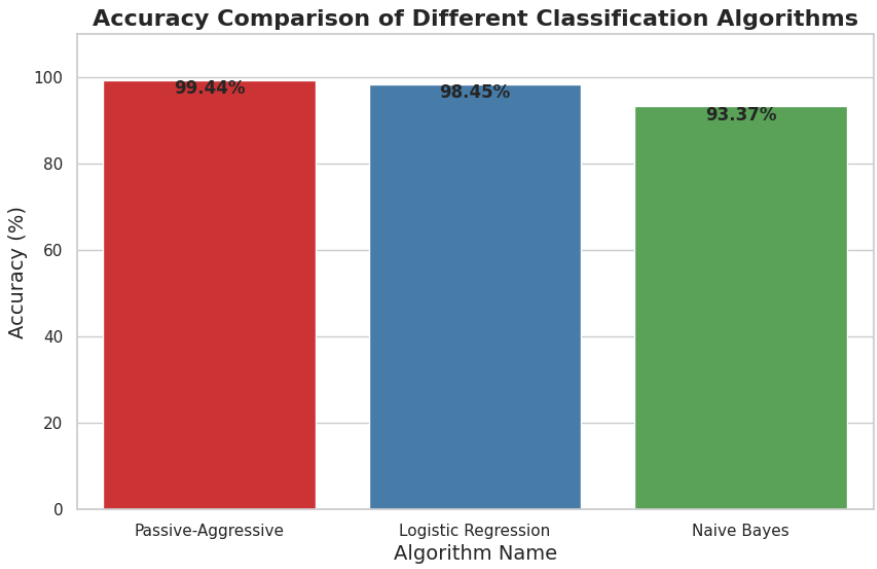


Fig. 3. Accuracy comparison of the classification algorithms

Fig. 3 compares the three models. The Passive-Aggressive Classifier is the most accurate at 99.44 percent, about one percentage point ahead of Logistic Regression and six points ahead of Naive Bayes. Naive Bayes is the weakest, which is expected because its assumption that words are independent given the class is unrealistic for natural language.

Table 8. Classification report of the best model (Passive-Aggressive Classifier)

Class	Precision	Recall	F1-score	Support
FAKE	0.9949	0.9945	0.9947	4,710
REAL	0.9939	0.9944	0.9941	4,270
Accuracy			0.9944	8,980
Macro average	0.9944	0.9944	0.9944	8,980

Table 8 gives the class-wise measures, computed from the confusion matrix in Table 9 (the notebook report rounds all of them to 0.99). Precision, recall and F1 are almost identical for the two classes, so the model is not trading one class against the other.

Table 9. Confusion matrix of the Passive-Aggressive Classifier on the test set

True class	Predicted FAKE	Predicted REAL	Total
FAKE	4,684	26	4,710
REAL	24	4,246	4,270

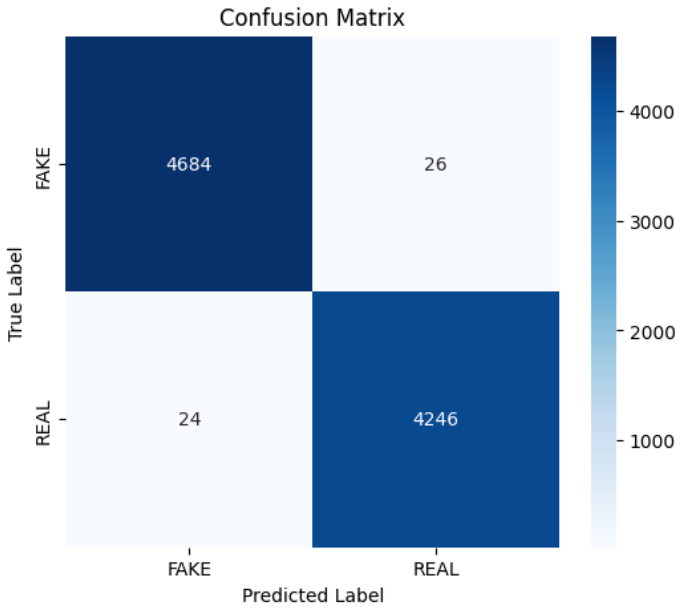


Fig. 4. Confusion matrix of the Passive-Aggressive Classifier on the test set

Fig. 4 shows that out of 8,980 test articles, 8,930 were classified correctly and 50 were wrong: 26 fake articles were accepted as real and 24 real articles were flagged as fake. The two error types are almost equal, which gives a balanced detector. The gaps between the models are large relative to this error count: Logistic Regression made roughly 139 errors and Naive Bayes roughly 595 on the same test set (estimated from their accuracies).

5.5 Interpretation and Comparison with Existing Approaches

Linear models on TF-IDF features are known to be strong baselines for text classification, and the Passive-Aggressive Classifier benefits from aggressive margin-based updates on very high-dimensional sparse data. The accuracy obtained here is higher than the roughly 92 percent reported with n-gram features and a linear SVM when the ISOT dataset was introduced [3]; differences in preprocessing, the use of the title, the split and the model all contribute, so the numbers should not be treated as a like-for-like comparison.

The very high scores must be read with care. Real articles in this dataset come from a single news agency and fake articles from many unreliable sites, so publisher style, formatting habits and recurring phrases can be learned as shortcuts. The model has therefore shown that it can distinguish the two sources in this collection, which is not the same as being able to verify the truth of a new claim from another outlet. Evaluation on a separate dataset such as LIAR [10], or a time-based split, would be needed to test generalisation.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of Major Findings

1. A pipeline of TF-IDF vectorisation and linear classifiers processed 44,898 news articles using sparse matrices, with no resampling needed because the classes are nearly balanced (23,481 fake and 21,417 real).
2. The Passive-Aggressive Classifier achieved the best test accuracy of 99.44 percent, with precision, recall and F1-score of about 0.994 for both classes.
3. Logistic Regression reached 98.45 percent and Multinomial Naive Bayes 93.37 percent, so the choice of algorithm changed the accuracy by up to six percentage points on identical features.
4. The best model made only 50 errors on 8,980 test articles, split almost evenly between fake news accepted as real (26) and real news flagged as fake (24).
5. Linear online learners offer an excellent balance of accuracy, speed and memory use, which makes them attractive for large-scale and streaming fake news screening.

6.2 Limitations

1. The ISOT dataset covers mainly political news from 2016 and 2017 and draws real articles from a single agency, so results may not generalise to other topics, outlets or periods.
2. The split is random rather than chronological, and similar stories can appear in both the training and test sets, which can inflate scores.
3. Source-specific wording and formatting can act as shortcuts, so the model may be recognising publisher style rather than truthfulness.
4. Content-only features cannot verify facts, and word-frequency models ignore word order and context.
5. The results come from a single split and a single run; cross-validation, repeated splits and hyper-parameter tuning were not carried out, and Logistic Regression and Naive Bayes were evaluated on accuracy only.
6. At about 45,000 articles the corpus fits easily in the memory of one machine; the scalability claims are by design and were not tested on a cluster.

6.3 Scope for Future Work

1. Evaluate on other datasets such as LIAR and on a time-based split to measure generalisation.
2. Add stylistic, sentiment and metadata features, and n-gram or character-level TF-IDF representations.

3. Compare with SVM, tree ensembles and transformer models such as BERT, and add cross-validation and hyper-parameter tuning.
4. Distribute the pipeline with Apache Spark MLlib and use incremental learning for continuously arriving articles.
5. Combine content with social-context signals such as propagation patterns and user credibility.
6. Build a deployed service or browser extension that scores articles in real time and explains its decision through the most influential words.

7. REFERENCES

[1] K. Shu, A. Sliva, S. Wang, J. Tang, and H. Liu, "Fake news detection on social media: A data mining perspective," ACM SIGKDD Explorations Newsletter, vol. 19, no. 1, pp. 22-36, 2017.

[2] S. Vosoughi, D. Roy, and S. Aral, "The spread of true and false news online," Science, vol. 359, no. 6380, pp. 1146-1151, 2018.

[3] H. Ahmed, I. Traore, and S. Saad, "Detection of online fake news using N-gram analysis and machine learning techniques," in Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments (ISDDC), LNCS vol. 10618, Springer, 2017, pp. 127-138.

[4] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, and Y. Singer, "Online passive-aggressive algorithms," Journal of Machine Learning Research, vol. 7, pp. 551-585, 2006.

[5] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[7] ISOT Fake News Dataset (Fake and Real News Dataset), University of Victoria / Kaggle. [Online]. Available: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

[8] V. Perez-Rosas, B. Kleinberg, A. Lefevre, and R. Mihalcea, "Automatic detection of fake news," in Proc. 27th International Conference on Computational Linguistics (COLING), 2018, pp. 3391-3401.

[9] B. D. Horne and S. Adali, "This just in: Fake news packs a lot in title, uses simpler, repetitive content in text body, more similar to satire than real news," in Proc. ICWSM Workshops, 2017.

[10] W. Y. Wang, "Liar, liar pants on fire: A new benchmark dataset for fake news detection," in Proc. 55th Annual Meeting of the ACL (Short Papers), 2017, pp. 422-426.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[12] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in AAAI-98 Workshop on Learning for Text Categorization, 1998, pp. 41-48.

[13] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in Proc. USENIX HotCloud, 2010.